

开源领域捐赠的实证研究数据实验

23020024何韬 23020040田子瀾 23020071毛恒

目录

一、引言	2
二、实验目的	2
三、实验设计	2
四、数据分析	4
五、运行过程	10
六、分析与结论	10

一、引言

开放源码软件无处不在，但持续发展却是一项挑战。开源软件在我们的软件基础设施、经济增长以及现代生活的几乎每一个方面都发挥着至关重要的作用，远远超出了技术软件项目的范畴：它几乎被用于每一种产品，或者被大大小小的公司用于创造产品的过程中，而开源软件的维护者往往看不到这些产品，或者根本没有做出任何贡献。

随着受欢迎程度的增加，维护和支持工作的需求通常也会增加包括大量的支持请求、功能请求和问题报告。维持开放源码是另一个重大挑战，社区已经意识到这一点，并正在进行有争议的讨论。支持开源的模式五花八门，包括依赖经常超负荷工作的志愿者、企业和基金会的赞助、销售高级版本、托管服务和支持、通过出书、咨询或演讲募集资金，以及简单地要求捐款。

本次研究重点关注捐赠和众筹平台，如 PayPal、Patreon 和 OpenCollective。许多社区博客文章和播客、GitHub 最近新增的赞助商功能显著且统一地支持捐赠请求，以及实验发现都证明，捐赠作为一种潜在的可持续可行模式在开源领域越来越受欢迎。尽管捐赠在支持开源活动方面似乎获得了牵引力，但人们对其普遍性、成功率和影响却知之甚少。

通过一项探索性的混合方法实证研究，调查了开放源代码中捐赠的目标、现实和使用情况，有助于加深对这一新现象的理解，并提供了数据。这份研究报告不仅为我们提供了对这个专业社区的深入理解，还为我们理解和预见软件开发行业未来的生态格局提供宝贵的洞察视角。

二、实验目的

开放源代码无处不在，许多项目充当着重要基础设施的角色，然而，为整个生态系统提供资金并使其持续发展却极具挑战性。虽然开源项目有许多不同的筹资模式，基金会也在共同努力，但 PayPal、Patreon 和 OpenCollective 等捐赠平台是为开源开发筹集资金的流行且门槛较低的平台。通过一项混合方法研究，调查了开源领域新兴的、在很大程度上尚未被探索的捐赠现象。具体而言，量化了开源项目请求捐款的普遍程度，统计了请求和接受捐款的项目特征，分析请求资金的需求和用途，并评估了收到的捐款是否达到了预期效果。

三、实验设计

本次实验基于GitHub已有的25,885个项目要求捐款（通常是为了支持工程活动）。采用的研究方法如下：

频率和常用平台：通过大规模的代码库挖掘，进行全面普查以了解有多少开源项目发起了募捐请求，并确定常用的募捐平台；

提问和接受捐赠的项目特征：通过多元回归模型研究更有可能发起募捐请求的项目的特征以及在募捐方面取得成功的项目的特征；

(1) 对募捐期望的分析：通过定性文本分析技术分析开发者发起募捐请求的原因，生成可验证的关于捐赠期望效果的假设；

(2) 可观察的捐赠结果：通过中断时间序列设计模型，对成功筹集到捐赠的项目进行长期的捐赠效果分析，以验证捐赠是否会带来活动增长等可观察的结果；

(3) 捐赠的使用情况：在捐赠结果方面得出较弱的结论后，通过定性分析探究捐赠期望与实际结果之间的区别，重点关注项目中的捐赠使用情况。

本实验使用了两个数据集进行研究。第一个是npm包对应的所有GitHub代码仓库，总计537,640个，第二个是截至2019年5月23日的所有GitHub代码仓库，总计77,934,441个。这两个数据集被用来研究开源项目的募捐情况和捐赠效果。

 asking_group_npm.csv	2020/2/19 13:21	XLS 工作表	584 KB
 asking_group_npm_gh.csv	2020/2/19 13:21	XLS 工作表	12,376 KB
 asking_money_sample.csv	2020/2/19 13:21	XLS 工作表	4 KB
 asking_no_money_sample.csv	2020/2/19 13:21	XLS 工作表	8 KB
 asking_some_money_sample.csv	2020/2/19 13:21	XLS 工作表	5 KB
 commercial_npm_control.csv	2020/2/19 13:21	XLS 工作表	582 KB
 highest_download_count_npm_control.csv	2020/2/19 13:21	XLS 工作表	1,644 KB
 random_gh_control.csv	2020/2/19 13:21	XLS 工作表	22,729 KB
 random_npm_control.csv	2020/2/19 13:21	XLS 工作表	3,089 KB
 rdd_ask_funding_date.csv	2020/2/19 13:21	XLS 工作表	3,960 KB
 rdd_ask_funding_date_monthly_earning.csv	2020/2/19 13:21	XLS 工作表	1,290 KB
 rdd_get_funding_date.csv	2020/2/19 13:21	XLS 工作表	1,002 KB
 rdd_get_funding_date_monthly_earning.csv	2020/2/19 13:21	XLS 工作表	316 KB

图1 数据集文件

数据文件夹包含以下 CSV 文件

- asking_group_npm_gh.csv: 要求捐款的 GitHub 项目的度量指标
- asking_group_npm.csv: 请求捐款的 npm 项目的指标
- asking_money_sample.csv: 接受大量捐赠的项目样本，用于定性分析
- asking_no_money_sample.csv: 未收到任何捐赠的项目样本，用于定性分析
- asking_some_money_sample.csv: 收到一些捐赠的项目样本，用于定性分析
- commercial_npm_control.csv: 由商业机构拥有且不要求捐赠的 npm 项目的指标
- highest_download_count_npm_control.csv: 下载次数最高且不要求捐赠的 npm 项目的指标值
- random_gh_control.csv: 不要求捐赠的 GitHub 项目的随机样本指标

- random_npm_control.csv: 不要求捐款的 npm 项目随机样本的指标
- rdd_ask_funding_date_monthly_earning.csv: 要求捐款的项目的月收入数据
- rdd_ask_funding_date.csv: 要求捐款的项目的时间序列数据
- rdd_get_funding_date_monthly_earning.csv: 接受捐赠项目的月收入数据
- rdd_get_funding_date.csv: 接受捐赠项目的时间序列数据

四、数据分析

实验从一份流行的、人工策划的开源筹资模式列表开始，筛选出 11 个捐赠和众包平台，包括 PayPal、Patreon 和 Liberapay。然后，对使用了这些服务的项目进行了人工抽样，并检查了它们的 GitHub 资源库，以寻找表明它们正在使用这些服务的信号，我们注意到，在这些资源库的 README.md 文件中，几乎总是以链接的形式引用了这些筹资平台。接下来，反复整理了一份捐赠平台列表和相应的搜索字符串，以便在标记符中识别它们。

为了超越最初的 11 个平台，实验使用了两种策略：首先，通过网络搜索捐赠平台。其次，使用 "donate"、"sponsor" 和 "donation" 等术语，对 npm 语料库中所有 537,640 个版本库的所有标记符进行了大规模的 grep 搜索；然后，从 8093 个 npm 版本库中随机抽取了 367 个包含此类术语的版本库（在 95% 置信度和 5% 置信区间下的代表性样本），对其中遗漏的平台进行了人工调查。发现了两种无法自动准确检测的捐赠形式，因此将其排除在分析之外：提及赞助商但没有明确的赞助管理平台，以及偶尔使用比特币但以多种不同方式表述，因此尝试的所有搜索模式要么只能识别这些软件源的一小部分子集，要么产生太多误报而无用。的最终列表包含 12 个捐赠平台，它们都能在标记符中自动检测到：Bountysource、Flattr、IssueHunt、Kickstarter、Liberapay（包括 Gittip 和 Gratipay）、OpenCollective、Otechie、Patreon、PayPal、SALT、Tidelift 和 Tip4Commit。

因此，绘制了 npm 社区和更大的 GitHub 社区中申请资助的项目数量随时间变化的曲线图。

可视化代码如下：

```
df_all = pd.read_csv('../data/asking_group_npm_gh.csv')
df_npm = pd.read_csv('../data/asking_group_npm.csv')
```

总体而言，要求捐赠的项目很多，但只占我们语料库的一小部分：实验发现，在2019年截止日期，npm 语料库的 537,640 个软件源中有 1,145 个（0.2%）要求捐赠。在 GitHub 语料库中，要求捐赠的项目比例更低，77,934,441 个软件源中有 25,885 个（0.04%）。如下图所示，按 npm 存

```
def plot_time_data(df, formaty=True):
    df_time1 = pd.DataFrame(columns=['date', 'count', 'service'])
    services = ['paypal', 'patreon', 'flattr', 'liberapay', 'opencollective', 'other']
    for service in services:
        df_time1 = df_time1.append(get_time_data(service, df), ignore_index=True)
    markers = {'paypal': 'o', 'patreon': '^', 'opencollective': '|', 'flattr': '|', 'liberapay': '|', 'other': '|'}
    linestyle = {'paypal': 'solid', 'patreon': 'solid', 'opencollective': 'solid', 'flattr': 'solid', 'liberapay': 'dashed', 'other': 'dotted'}
    hues = {'paypal': '#5D7EA7', 'patreon': '#EFC6D1', 'flattr': '#85BB65', 'liberapay': '#C58293',
            'opencollective': '#9FCDCD', 'other': '#3E3E3C'}
    plt.figure(figsize=(10, 5))
    for service, df_service in df_time1.groupby('service', sort=False):
        plt.plot(df_service['date'].values, df_service['count'].values, label=service, linewidth=2.5,
                 marker=markers[service], markersize=7, color=hues[service], linestyle=linestyle[service], markevery=3)
    if formaty:
        formatter = FuncFormatter(thousands)
        plt.gca().yaxis.set_major_formatter(formatter)
    plt.legend(loc='upper left', prop={'size': 20}, frameon=False)
    plt.ylabel('Count', fontsize=22, labelpad=15)
    plt.xlim([convert_datetime('2008-01-01'), convert_datetime('2019-05-23')])
    plt.rc('xtick', labels=22)
    plt.rc('ytick', labels=22)
    sns.despine()
    return plt
```

储库数量计算，Patreon 是最常见的捐赠平台，其次是 PayPal 和 OpenCollective，其他服务的采用率要低得多。

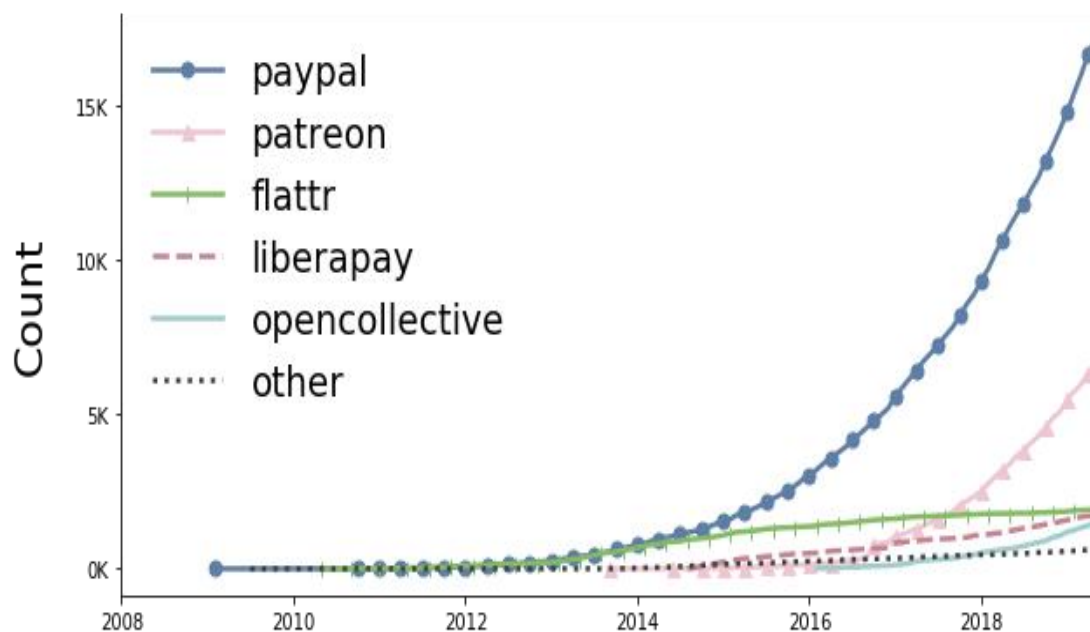


图2 GitHub 社区申请项目数量

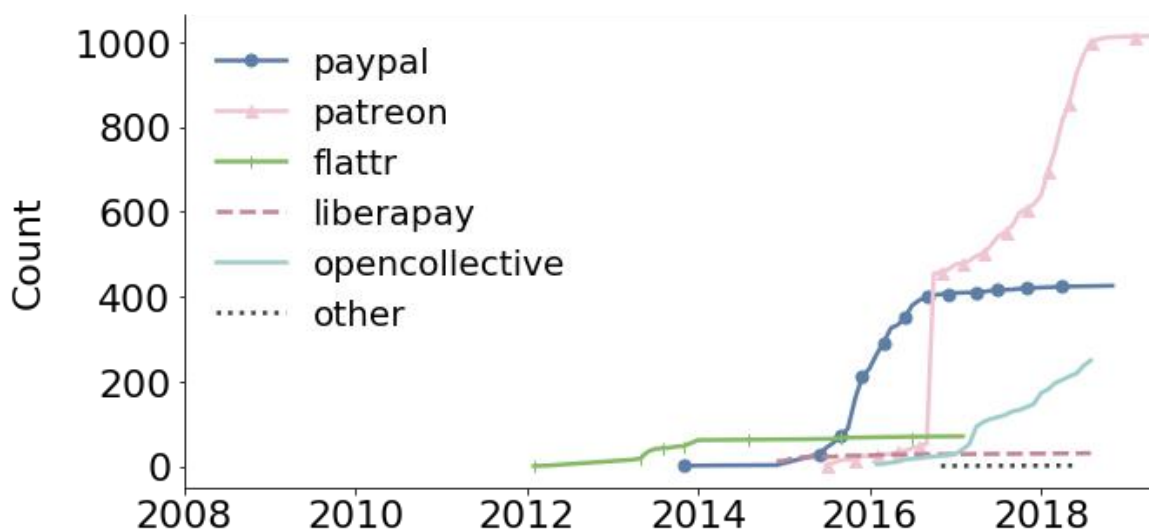


图3 npm社区申请项目数量

为了探究捐款的可观察效果有限的原因，实验对单个项目进行了更深入的分析，发现不同项目对捐款的使用（在公开可见的情况下）存在显著差异。

在对 15 个高捐款项目和中等捐款项目进行了一些探索性分析后，发现使用 OpenCollective 的项目能提供最多的启示，因为这些项目的捐款情况通常最为详细，而且它们还包括一份公开的支出清单。

建立了一个新的语料库，其中包含 60 个（仅）使用 OpenCollective 募捐的项目。从接受捐赠最多的 15 个 npm 项目开始。为了实现语料库的多样性，采用分层抽样的方法增加了 45 个项目：将 GitHub 语料库中的 OpenCollective 项目分为三个阶层：过去 9 个月中平均每月捐款超过 1000 美元的项目、每月捐款少于 100 美元的项目以及介于两者之间的项目，每个阶层随机抽取 15 个项目。首先自动为每个项目收集摘要描述。每个描述都包含从 OpenCollective 收集到的交易历史和资金数额的数据和图表。然后，通过手动调查每个项目的公开人工制品，用注释丰富了自动汇总的摘要。我们仔细阅读了项目的 README.md 文件、网页、文档和捐赠页面，并调查了其捐赠历史（收入和支出）以及 GitHub 在版本库和贡献者层面的活动图表。为了指导调查捐款的使用情况，试图收集项目要求捐款的原因、资金的接收者以及资金的用途等信息。在收集了所有 60 个项目的原始观察结果后，寻找数据中的模式并讨论观察结果的潜在影响。然后，在可能的情况下，对特定观察结果（例如，接受但未使用捐款的项目，见下文）进行了操作化，以量化这些观察结果在 GitHub 语料库中所有 540 个项目中的出现频率，这些项目在截止日期 2019 年 5 月 23 日之前的 9 个月中通过 OpenCollective 接受了捐款。有效性和可信度的威胁。

很多人都在为两个项目省钱和花钱，一个是省钱的，另一个是花钱的。实验将按照定性分析中出现的主题讨论我们的观察结果，储蓄者与支出者。

可视化代码如下：

```
def plot_oc_data(url_name):
    plt.clf()
    path = '../data_mining/files/20190719_OpenCollective CSV'
    directory = os.fsencode(path)
    for file in os.listdir(directory):
        # Open csv file
        file = file.decode("utf-8")
        name = file.split('--')[0]
        if name == url_name.lower():
            df_oc = pd.read_csv(path+'/'+file)
            df_oc = df_oc.iloc[::-1]
            (dates, gains, expenses, balances) = get_data(df_oc)
            x = dates
            y = [expenses, np.subtract(gains, expenses)]
            pal = ['#85BB65', '#E8E9C9']
            plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m'))
            plt.gca().xaxis.set_major_locator(mdates.MonthLocator())
            formatter = FuncFormatter(thousands)
            plt.gca().yaxis.set_major_formatter(formatter)
            plt.stackplot(x, y, labels=['Spent', 'Saved'], colors=pal)
            plt.legend(loc='upper left', prop={'size': 22}, frameon=False)
            plt.margins(0.005, 0)
            plt.gca().xaxis.set_major_locator(plt.MaxNLocator(10))
            plt.rc('xtick', labels=22)
            plt.rc('ytick', labels=22)
            plt.xticks(rotation=27)
            fig = plt.gcf()
            fig.set_size_inches(7, 3)
            sns.despine(fig=fig)
```

发现，在支出方面存在很大差异：一些项目积极使用所有筹集到的资金，而另一些项目则几乎不使用任何资金，有时还会积累大量资金--可视化图显示了两个例子。在我们的样本中，有 24 个项目（40%）花费了不到 25% 的募捐资金，有 9 个项目（15%）花费了 75% 以上的募捐资金。在对 GitHub 语料库中的所有 OpenCollective 项目进行自动分析时，我们发现节省开支是普遍现象，64% 的项目花费的资金少于所获捐款的 25%，只有 11% 的项目花费的资金超过 75%。



图4 saved项目示例

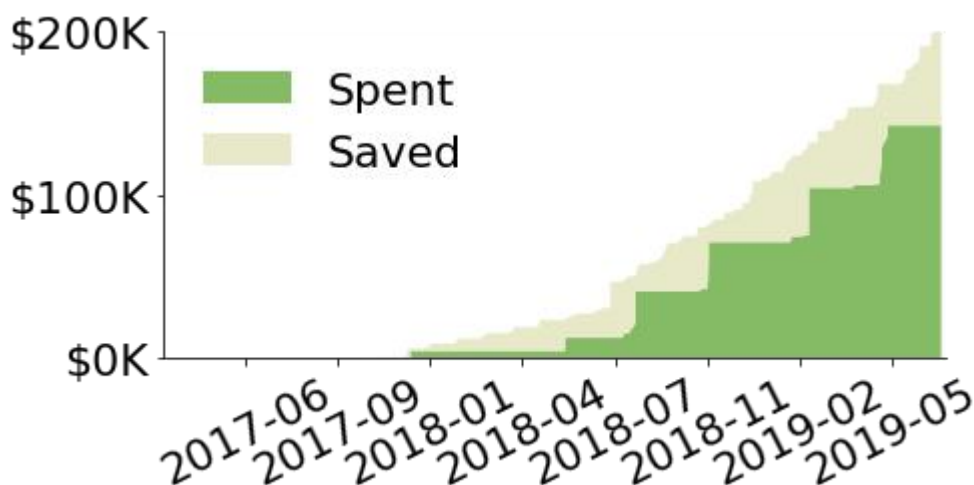


图5 花钱项目示例

在 21 个有明确请求捐款理由和明确支出的项目中，只有一个项目出现了差异：该项目请求捐款以支持全职维护者和增加工具支持，但却只将钱花在了品牌推广和差旅上；然而，其捐款收入和支出都相当低，因此在当前的资金水平下，资助工程设计可能并不可行。为了在所有使用 OpenCollective 收集捐款的 GitHub 项目中自动进行这一分析，我们手动开发了一个分类器，使用关键字匹配将交易描述分为工程、服务（如服务器）、差旅/食品（如会议费）、对其他项目的捐款和社交（如商品）等类别。

实验结果如下：

	slug	url	earning_last9months	total_expense	Engineering	Services	Travel/Food	Donations	Social	Unclassified	Engineering_frac	Services_frac	Travel/Food_frac	Donations_frac	Social_frac	Unclassified_frac
0	checkstyle/checkstyle	checkstyle	20.00	0	0	0	0	0	0	0	0.000000	0.000000	0.0	0.000000	0.000000	0.000000
1	clojure-emacs/cider	cider	9283.00	5063.64	4950	113.64	0	0	0	0	97.755765	2.244235	0.0	0.000000	0.000000	0.000000
2	uraimo/awesome-software-patterns	duplication	2563.00	0	0	0	0	0	0	0	0.000000	0.000000	0.0	0.000000	0.000000	0.000000
3	NaturalIntelligence/imglab	imglab	0.00	0	0	0	0	0	0	0	0.000000	0.000000	0.0	0.000000	0.000000	0.000000
4	joshtronic/donate	mochajs	18062.49	3576.9	2563.94	93.96	0	502	67	350	71.680505	2.626856	0.0	14.034499	1.87313	9.785009

按资金水平划分的细目支持了我们的观察，即资金水平高的项目主要将资金用于工程，而资金水平低的项目将较大比例的资金用于服务。实现的可视化如下

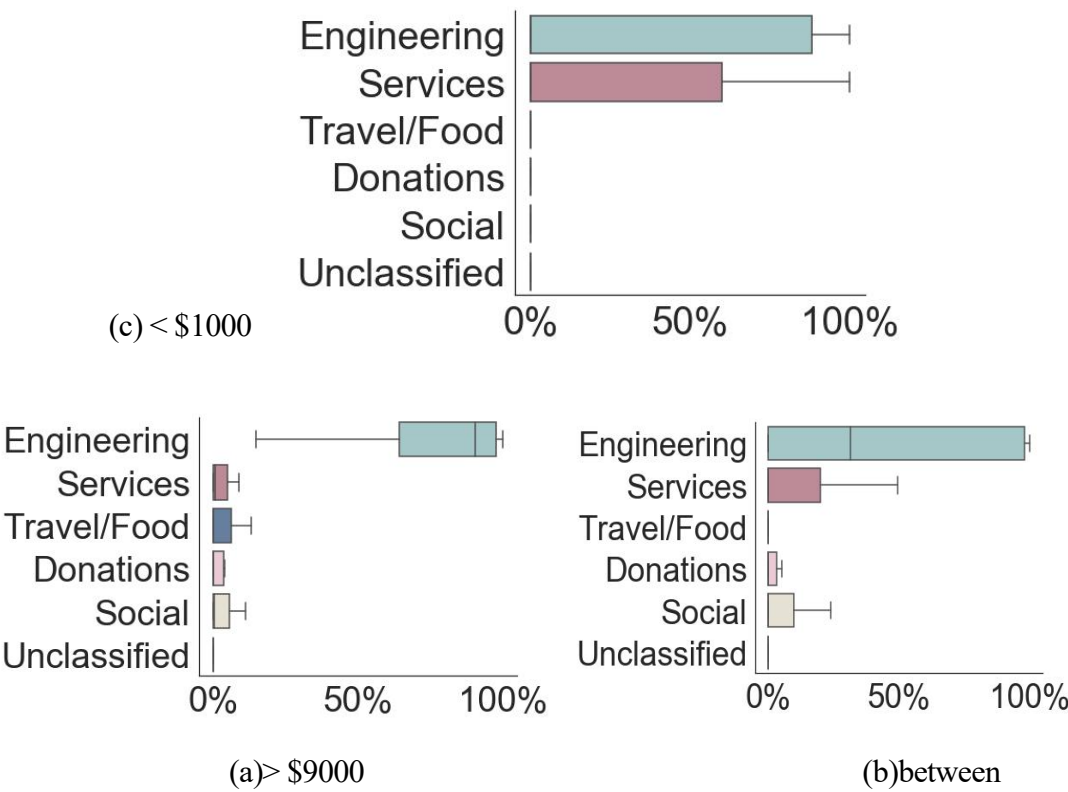
可视化代码：

```
def plot_boxplot(df):
    figsize = (6, 5)
    euro_colors = ['#9FCDCD', '#C58293', '#5D7EA7', '#EFC6D1', '#E7E4D3', '#85B865']
    sns.set(style="white", rc={'figure.figsize':figsize, 'xtick.labelsize':37, 'ytick.labelsize':37})
    ax = sns.boxplot(x='frac', y='type', data=df, showfliers=False, palette=euro_colors, orient="h")
    ax.set(xlabel='', ylabel='')
    formatter = FuncFormatter(percents)
    plt.gca().xaxis.set_major_formatter(formatter)
    sns.despine()
    fig = ax.get_figure()
    return fig

df_money = df_expenses2[df_expenses2.earning_last9months >= 9000]
df_money_dummy = pd.DataFrame(columns=['type', 'frac'])
for index, row in df_money.iterrows():
    if row['total_expense'] == 0:
        continue
    for typ in types:
        df_money_dummy = df_money_dummy.append({'type':typ, 'frac':row[typ+'_frac']], ignore_index=True)

df_some_money = df_expenses2[(df_expenses2.earning_last9months < 9000)&(df_expenses2.earning_last9months >= 1000)]
df_some_money_dummy = pd.DataFrame(columns=['type', 'frac'])
for index, row in df_some_money.iterrows():
    if row['total_expense'] == 0:
        continue
    for typ in types:
        df_some_money_dummy = df_some_money_dummy.append({'type':typ, 'frac':row[typ+'_frac']], ignore_index=True)
```

分别实现了OpenCollective 交易类别在三个资助级别的 GitHub 项目中的分布情况。如下图所示：



部分实验结果如下图存放在文件里。

issue-speed	2024/2/26 8:55	文件夹	
who-asks-money-gh	2024/2/26 8:55	文件夹	
who-asks-money-npm	2024/2/26 8:55	文件夹	
who-gets-money-gh	2024/2/26 8:55	文件夹	
who-gets-money-npm	2024/2/26 8:55	文件夹	
20190627_patreon_stats.csv	2024/2/27 8:51	XLS 工作表	247 KB
20190719_OpenCollective CSV	2024/2/27 8:47	文件	83 KB
20190727_oc_adoption_dates.csv	2024/2/27 8:53	XLS 工作表	72 KB
20190730_patreon_adoption_dates.csv	2024/2/27 8:55	XLS 工作表	222 KB
20190821_get_dates.csv	2024/2/27 10:06	XLS 工作表	1,090 KB

五、运行过程

```
A:\oss-donations-master\src\data_mining\get_funding_adoption_dates_opencollective.py:23: DtypeWarning: Columns (44,45,46,47,48,49,62,63,64) have mixed types. Specify dtype option on import or set low_memory=False.
df_data = pd.read_csv('files/20190727_asking_all.csv')
100% | 540/540 [00:02<00:00, 206.46it/s]
A:\oss-donations-master\src\data_mining\get_funding_adoption_dates_patreon.py:41: DtypeWarning: Columns (44,45,46,47,48,49,62,63,64) have mixed types. Specify dtype option on import or set low_memory=False.
df_data = pd.read_csv('files/20190727_asking_all.csv')
100% | 2412/2412 [00:40<00:00, 59.51it/s]
A:\oss-donations-master\src\data_mining\get_funding_amount_opencollective.py:15: DtypeWarning: Columns (44,45,46,47,48,49,62,63,64) have mixed types. Specify dtype option on import or set low_memory=False.
df_data = pd.read_csv('files/20190727_asking_all.csv')
100% | 1756/1756 [00:14<00:00, 125.33it/s]
A:\oss-donations-master\src\data_mining\get_funding_amount_patreon.py:12: DtypeWarning: Columns (44,45,46,47,48,49,62,63,64) have mixed types. Specify dtype option on import or set low_memory=False.
df_data = pd.read_csv('files/20190727_asking_all.csv')
100% | 7005/7005 [01:17<00:00, 90.87it/s]
100% | 5811/5811 [00:01<00:00, 5630.41it/s]
```

六、分析与结论

实验结果表明采用混合方法探讨了开源项目中的捐赠状况。发现，只有一小部分 npm 软件包和 GitHub 存储库要求捐款，最常用的捐款平台是 PayPal 和 Patreon。要求和接受捐赠的项目往往更活跃、更成熟、更受欢迎。在要求捐款时，开发者通常表示他们希望支持代码和非代码工程活动，但也提到了支付运营费用、促进社区发展和减轻压力。为了探究预期与结果之间似乎存在的不匹配，研究了募集资金的使用情况，结果发现，募集资金的使用通常与申请目的相符（如果募集到足够的资金），但许多项目实际上只 使用了其资金的一小部分，而且，资助最高捐款者之外的其他捐款者的情况也不少见。